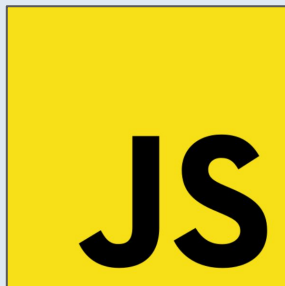


Mòdul 06



Institut TIC
de Barcelona



RA3.FE - Objectes

- Tipus de funcions
- Objectes
- Objecte windows
- BOM
- Emmagatzematge
 - cookies
 - session storage
 - local storage
- BroadCast channel

JavaScript és un **paradigma d'objectes** perquè gairebé TOT es tracta o es comporta com un objecte.

És un llenguatge dissenyat al voltant del concepte d'objectes.

- Arrays, Dates, etc. són objectes
- Les Funcions es comporten com a objectes
- El DOM treballa amb objectes
- JSON vol dir JavaScript Object Notation

Function declaration

- Li definim un nom a la funció.
- Es poden cridar abans de declarar-la.

```
function calcularEdat(dataNaixement) {  
  return 2024 - dataNaixement;  
}
```

Function expression

- Funció que es guarda en una variable.
- Normalment s'utilitzen amb **funcions anònimes (funcions sense nom)**.
- No es poden cridar abans de declarar-la.

```
const calcularEdat = function (dataNaixement) {  
  return 2024 - dataNaixement;  
};
```

Les 2 funcions es criden igual i donen el mateix resultat:

```
calcularEdat(1990);
```

- Un objecte és una col·lecció de propietats i tipus.
- Una propietat és l'associació *key : value*
- Un *value* pot ser:
 - *Variables* de tipus primitives (*number, string, boolean, undefined, ...*)
 - *Objectes*: *objectes, arrays, dates,...*
 - *Funcions i expressions*

```
const persona = {  
  nom: "Jhon",  
  cognom: "Smith",  
  edat: 33,  
  teCarnetConduir: true,  
  altura: 1.79,  
};
```

Forma més actual de
definir objectes

```
const persona = new Object();  
  
persona.nom = "Jhon";  
persona.cognom = "Smith";  
persona.edat = 33;  
persona.teCarnetConduir = true;  
persona.altura = 1.79;
```

- Tipus de propietats

```
const persona = {  
  nom: "Jhon",  
  cognom: "Smith",  
  teCarnetConduir: true,  
  altura: 1.79,  
  edat: 2024 - 1999,  
  amics: ["Marta", "Joan"],  
  direccio: {  
    ciutat: "Barcelona",  
    cp: "09005",  
  },  
  esMajorEdat: function () {  
    return 2024 - 1999 > 18;  
  },  
};
```

expressions i operacions

array

objecte

function expression

- Com consultar les propietats

```
const persona = {  
  nom: "Jhon",  
  cognom: "Smith",  
  teCarnetConduir: true,  
  altura: 1.79,  
  edat: 2024 - 1999,  
  amics: ["Marta", "Joan"],  
  direccio: {  
    ciutat: "Barcelona",  
    cp: "09005",  
  },  
  esMajorEdat: function () {  
    return 2024 - 1999 > 18;  
  },  
};
```

```
// objectName.property  
console.log(persona.nom);  
console.log(persona.edat);  
console.log(persona.amics[1]);  
console.log(persona.direccio.ciutat);  
console.log(persona.esMajorEdat());
```

Jhon
25
Joan
Barcelona
true

```
//objectName["property"]  
console.log(persona["nom"]);  
console.log(persona["edat"]);  
console.log(persona["amics"][1]);  
console.log(persona["direccio"].ciutat);  
console.log(persona["direccio"]["cp"]);  
console.log(persona["esMajorEdat"]());
```

Jhon
25
Joan
Barcelona
09005
true

```
//objectName[expression]  
const varEdat = "edat";  
console.log(persona["no" + "m"]);  
console.log(persona[varEdat]);
```

Jhon
25

- Com consultar les propietats

```
const persona = {  
  nom: "Jhon",  
  cognom: "Smith",  
  teCarnetConduir: true,  
  altura: 1.79,  
  edat: 2024 - 1999,  
  amics: ["Marta", "Joan"],  
};
```

```
//Mostrar les propietats amb un bucle  
let text = "";  
for (let x in persona) {  
  text += persona[x] + " ";  
}
```

Jhon Smith true 1.79 25 Marta,Joan

for (let x in object)

- Afegir propietats

Als objectes creats per nosaltres, tal i com s'han explicat a les diapositives anteriors, els hi podem afegir propietats sempre que sigui necessari inclus després de tenir-lo creat.

Però hi haurà casos en que els objectes depenent de com estiguin configurats no els hi podrem afegir propietats.

- La propietat *THIS*

```
const persona = {  
  nom: "Jhon",  
  cognom: "Smith",  
  teCarnetConduir: true,  
  altura: 1.79,  
  edat: 2024 - 1999,  
  amics: ["Marta", "Joan"],  
  direccio: {  
    ciutat: "Barcelona",  
    cp: "09005",  
  },  
  esMajorEdat: function () {  
    return 2024 - 1999 > 18;  
  },  
};
```

Les dos expressions
serveixen per definir
l'edat.
S'hauria d'evitar aquests
casos!!
Podem utilitzar **this** per
utilitzar les propietats
dins del mateix objecte.

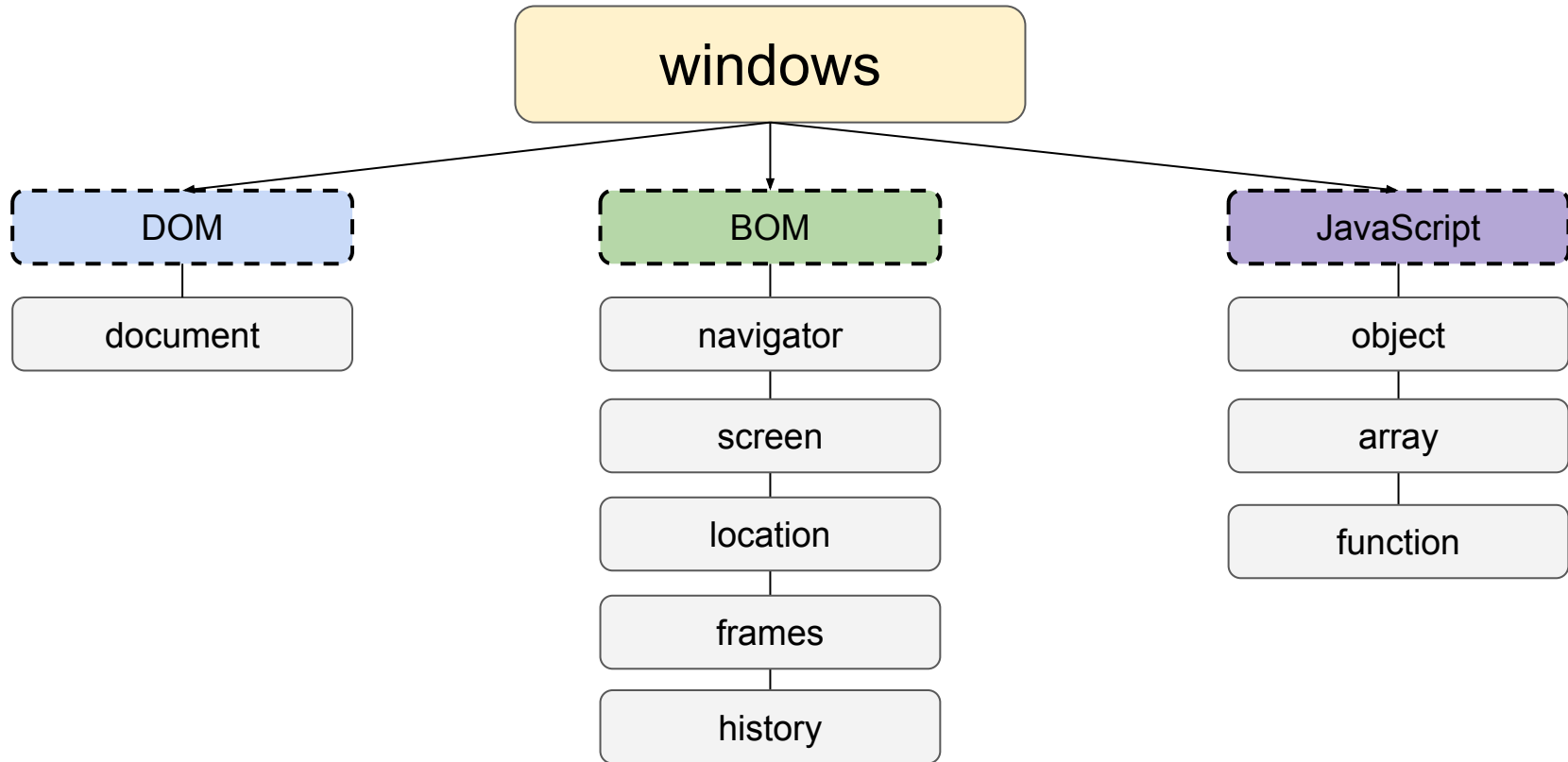
```
const persona = {  
  nom: "Jhon",  
  cognom: "Smith",  
  teCarnetConduir: true,  
  altura: 1.79,  
  edat: 2024 - 1999,  
  amics: ["Marta", "Joan"],  
  direccio: {  
    ciutat: "Barcelona",  
    cp: "09005",  
  },  
  esMajorEdat: function () {  
    return this.edat > 18;  
  },  
};
```

- És l'objecte global i principal en l'entorn del navegador web.
- Cada finestra o pestanya té el seu propi objecte window.
- Aquest objecte conté:
 - Tots els objectes, funcions, variables,... de javascript.
 - Les propietats del DOM (Document Object Model)
 - Les propietats del navegador (BOM. Browser Object Model)

```
window.document.getElementById("header");
```

=

```
document.getElementById("header");
```



Mètodes més destacats

- **alert(missatge):** crea una finestra de diàleg on es mostra el missatge.
- **confirm(missatge):** crea una finestra de confirmació (OK/Cancel) on es mostra el missatge. Retorna true o false.
- **prompt(missatge):** crea una finestra de diàleg on es mostra el missatge i permet l'edició.
- **open(url, windowName, params):** crea una finestra nova, li associa el nom, i accedeix a la URL que li hem passat.
- **close():** atura la càrrega de la finestra.

```
window.open("instruccions.html", "Instruccions", "width=400,height=400");  
window.close();  
const win = window.open("joc.html", "_blank").focus();  
win.close();
```

Permet consultar la **informació del navegador** com l'idioma, el tipus de navegador,...

Propietats més destacades:

- **navigator.language:** retorna l'idioma seleccionat en el navegador.
- **navigator.userAgent:** retorn la informació del navegador.

```
const sUsrAg = navigator.userAgent;  
const lan = navigator.language;
```

Permet consultar **informació de la URL**.

Propietats més destacades:

- **href**: retorna la URL sencera.
- **origin**: retorna el protocol, hostname i port de la URL.
- **pathname**: retorna el path de la URL.
- **port**: retorna el número de port de la URL.
- **protocol**: retorna el protocol de la URL.
- **host**: retorna el hostname i el número de port de la URL.
- **hostname**: retorna el hostname de la URL.

```
const href = location.href;           //http://127.0.0.1:5500/joc.html
const pathname = location.pathname;   //joc.html
const origin = location.origin;       //http://127.0.0.1:5500
const hostname = location.hostname;   //127.0.0.1
const host = location.host;           //127.0.0.1:5500
const protocol = window.location.protocol; //http
```

Permet consultar **informació de la URL**.

Mètodes més destacades:

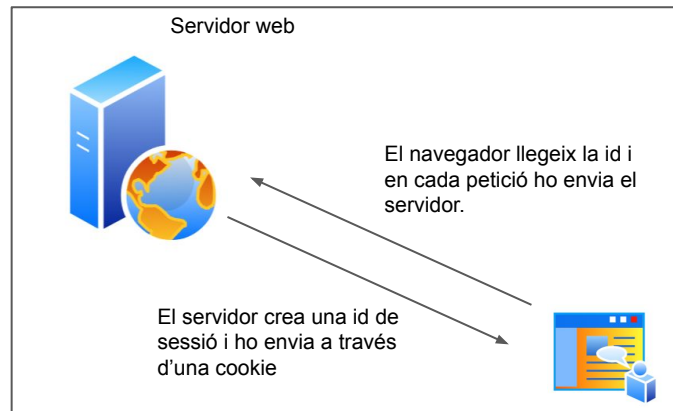
- **assign()**: carrega un nou document. Es manté a l'historial i l'usuari pot tornar a la pàgina anterior. També es pot fer servir ***window.location.href***
- **reload()**: torna a carregar la pàgina actual.
- **replace()**: reemplaça el document actual per un de nou. No es manté a l'historial i l'usuari no pot tornar a la pàgina anterior.

```
//Redireccionar a una nova pàgina  
window.location.assign("https://www.w3schools.com");
```

- Existeixen 3 formes d'emmagatzemar dades a nivell client:

	Cookies	LocalStorage	SessionStorage
Capacity	4kb	5-10 Mbs(Browser Dependent)	5 Mbs
Accessibility	All windows	All windows	Private to tab
Expiration	Manually Set	Never expires	On tab close.
Passed in request	Yes	No	No
Storage	Browser and Server	Browser Only	Browser Only

- Serveixen per poder emmagatzemar poca informació a la part del client:
 - Gestió de sessions (saber si un usuari té la sessió oberta)
 - Personalització (configuracions, preferències de l'usuari)
 - Rastreig (comportaments de l'usuari)
- Les cookies tenen una data de caducitat.
- S'envia a través de peticions http(s).



```
// Setting a cookie
document.cookie = "name=John; expires=Thu, 1 Jan 2023 00:00:00 UTC;
path="/";

// Removing a cookie
document.cookie = "name=; expires=Thu, 1 Jan 1970 00:00:00 UTC;
path="/";
```

```
// Get data cookie
function getCookie(cname) {
    let name = cname + "=";
    let decodedCookie = decodeURIComponent(document.cookie);
    let ca = decodedCookie.split(";");
    for (let i = 0; i < ca.length; i++) {
        let c = ca[i];
        while (c.charAt(0) == " ") {
            c = c.substring(1);
        }
        if (c.indexOf(name) == 0) {
            return c.substring(name.length, c.length);
        }
    }
    return "";
}
```

- Proporciona els mecanismes mitjançant els quals el navegador pot emmagatzemar informació de tipus clau/valor, de manera molt més intuïtiva que utilitzant cookies.
- Els mecanismes més habituals son:
 - *sessionStorage*
 - *localStorage*

- Serveixen per poder emmagatzemar uns 5 megas d'informació a la part del client.
- Les dades es guarden mitjançant el *key, value*. Els values son de tipus text.
- Les dades emmagatzemades persisteixen fins que es tanca la finestra o pestanya.

```
// Storing data in session storage
sessionStorage.setItem("name", "John");
sessionStorage.setItem("age", "30");
// Retrieving data from session storage
const name = sessionStorage.getItem("name");
const age = sessionStorage.getItem("age");
// Removing data from session storage
sessionStorage.removeItem("age");
// Clearing all data from session storage
sessionStorage.clear();
```

- Serveixen per poder emmagatzemar uns 5/10 megas d'informació a la part del client.
- Les dades es guarden mitjançant el *key, value*. Els values son de tipus text.
- Les dades emmagatzemades persisteixen fins i tot després que el navegador es tanqui i s'obri. S'ha d'eliminar la informació a través del codi.

```
// Storing data in local storage
localStorage.setItem("name", "John");
localStorage.setItem("age", "30");
// Retrieving data from local storage
const name = localStorage.getItem("name");
const age = localStorage.getItem("age");
// Removing data from local storage
localStorage.removeItem("age");
// Clearing all data from local storage
localStorage.clear();
```

- Com podem guardar i recuperar objectes en localStorage/sessionstorage

```
const config = {  
  lang:"",  
  userAgent: "",  
  url:"",  
  bgColorClass:""  
}
```

```
localStorage.setItem("config", JSON.stringify(config));  
const config = JSON.parse(localStorage.getItem("config"));
```

És una api del navegador que permet la comunicació bàsica entre diferents finestres o pestanyes que es trobin en el mateix origen (mateix protocol, hostname i port)

Ex mateix origen:

- <http://example.com/app1/index.html>
- <http://example.com/app2/index.html>

Ex diferent origen:

- <http://example.com/app1>
- <https://example.com/app2>

- <http://example.com>
- <http://www.example.com>
- <http://myapp.example.com>

- <http://example.com>
- <http://example.com:8080>

```
<!DOCTYPE html>
<html lang="en">
  <body>
    <h1>Pàgina que envia un missatge a una altre pàgina/h1>
    <button onclick="enviarMissatge()">Enviar missatge</button>
    <script>
      //Definim el BroadcastChannel amb el nom "my_channel"
      const broadcastChannel = new BroadcastChannel("my_channel");

      function enviarMissatge() {
        const message = "Missatge que envio a l'altre pestanya";

        //Mètode que ens proporciona l'API per poder enviar un missatge
        broadcastChannel.postMessage(message);
      }
    </script>
  </body>
</html>
```

Pàgina que envia un missatge a una altre pàgina

Enviar missatge

```
<!DOCTYPE html>
<html lang="en">
  <body>
    <h1>Pàgina que rep el missatge</h1>
    <p id="textEnviat"></p>
    <script>

      //Definim el BroadcastChannel amb el mateix nom "my_channel"
      const broadcastChannel = new BroadcastChannel("my_channel");

      // Propietat que ens proporciona l'API per poder rebre un missatge
      broadcastChannel.onmessage = (event) => {
        document.getElementById("textEnviat").textContent = event.data;
      };

    </script>
  </body>
</html>
```

Pàgina que rep el missatge



onclick del botó



Mateix origen
(protocol, hostname
i port)